Bahria University Lahore Campus

Department of Computer Science

Artificial Intelligence – BSCS 5

Assignment 2

UAV Fleet Planning with a Genetic Algorithm

Technical Report

Name: [Your Name]

Roll No: [Roll No]

1. Introduction

A company uses drones (UAVs) to survey a disaster area. We must decide where each drone flies, how fast it flies and how strong its sensor is. A good plan covers the important areas and finds the targets, but it should also save battery, finish quickly and stay away from dangerous zones. These goals conflict, so there is no single correct answer. I solved this problem with a Genetic Algorithm (GA) written from scratch in Python.

The area is a 50 x 50 grid. It has 3 no-fly zones (blocked cells), 3 hazard zones (high wind / radiation), 3 priority (disaster) zones and 15 targets. 3 drones start at the base (2.0, 2.0), visit 6 waypoints each and return to the base. Each drone may fly at most 200 cells. One hazard zone lies on top of the most important disaster zone, so the GA must find a balance between coverage and safety.

2. Chromosome Design

Each chromosome is one complete plan for the whole fleet. It is a real-valued array with one segment per drone. Every waypoint has four genes:

Table 1: Genes of one waypoint (chromosome size = 3 drones x 6 waypoints x 4 genes = 72 genes)

Gene	Meaning	Range
x, y	position of the waypoint	0 – 50
speed	speed on the way to this waypoint	1.0 – 3.0 cells/step
sensor power	sensor strength (radius 1 to 4 cells)	0 – 1

Why this design? (1) It is simple: the genes are exactly the things we must decide (waypoints, speed and sensor power). (2) Real numbers suit positions on a map: a small change in a gene gives a small change in the route, which works well with Gaussian mutation and blend crossover. (3) One segment per drone means a good route of one drone can be passed to a child as a whole block. (4) The sensor power gene lets the GA switch the sensor to low power where there is nothing to see and save energy.

3. Fitness Function

$F = 0.35 \cdot \text{Coverage} + 0.25 \cdot \text{Detection} - 0.15 \cdot \text{Energy} - 0.10 \cdot \text{Time} - 0.15 \cdot \text{Hazard} - 2 \cdot \text{Penalty}$

Table 2: Objectives and weights

Objective	How it is measured (scaled to 0–1)	Weight
Coverage (max)	priority-weighted share of cells inside a sensor circle	0.35
Detection (max)	share of targets inside a sensor circle	0.25
Energy (min)	distance x (1 + 0.3·speed) + sensor power x time, divided by the maximum	0.15
Time (min)	flight time of the slowest drone / longest possible time	0.1
Hazard (min)	average hazard value under the flown path	0.15

All objectives are scaled to 0–1, otherwise a large number (for example energy) would dominate. The two mission goals (coverage and detection) get 0.60 of the weight and the three costs get 0.40, so the drones are rewarded for doing the job and not for staying at the base. Coverage has the largest weight because surveying the disaster zones

is the main purpose. Hazard and energy both get 0.15 because drone safety and battery are equally important. Time gets the smallest weight because the range limit already keeps the missions short.

Constraint handling uses both methods. Repair: if a waypoint falls inside a no-fly zone, it is moved to the nearest open cell. Penalty: if a leg still flies through a no-fly zone, or a drone flies further than the maximum range, the fitness is reduced by 2 x (10 x share of the path inside no-fly zones + extra range). The penalty is large, so plans that break the rules cannot win.

4. Genetic Operators and Settings

Initial population: 50 random plans. Selection: tournament selection with k = 3 (pick k random plans, the best one becomes a parent); k controls the selection pressure. Crossover (rate 0.9): first a segment swap, where every drone's route is copied from parent 1 or parent 2; then a geometric blend, where one drone's waypoints are a random mix of both parents (a point between them). Mutation (rate 0.1 per waypoint): the waypoint moves by a small Gaussian step (about 3.0 cells), its speed and sensor power change a little, and sometimes two waypoints swap their order. To keep diversity, the 2 best plans survive unchanged (elitism) and 2 new random plans are added every generation. Termination: 100 generations, or no improvement for 30 generations.

5. Results of the Main Run

With seed 42 the run reached 100 generations (4850 fitness evaluations). The best fitness improved from -1.590 in generation 0 to 0.438. The final plan covers 0.89 of the priority-weighted area, finds 100% of the targets, has energy 0.50, time 0.36, hazard 0.09 and penalty 0.00 (no rules broken). The average fitness stays below the best because two random immigrants enter every generation.

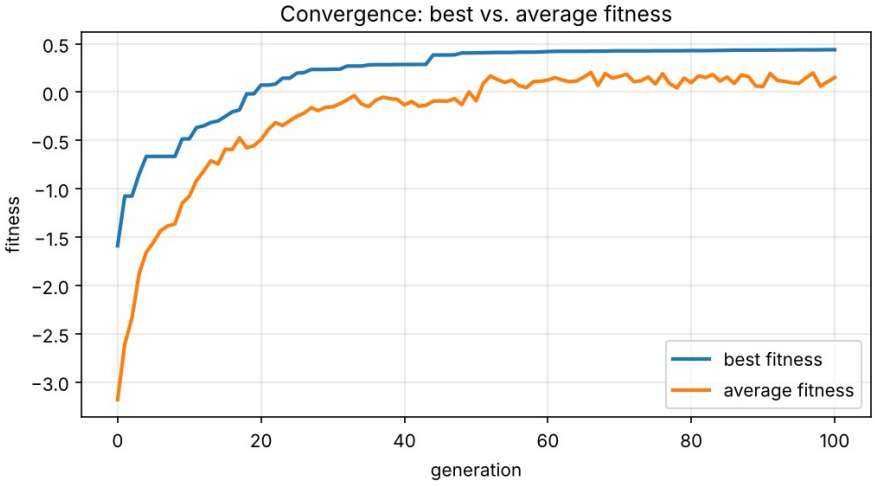


Figure 1: Best and average fitness over the generations

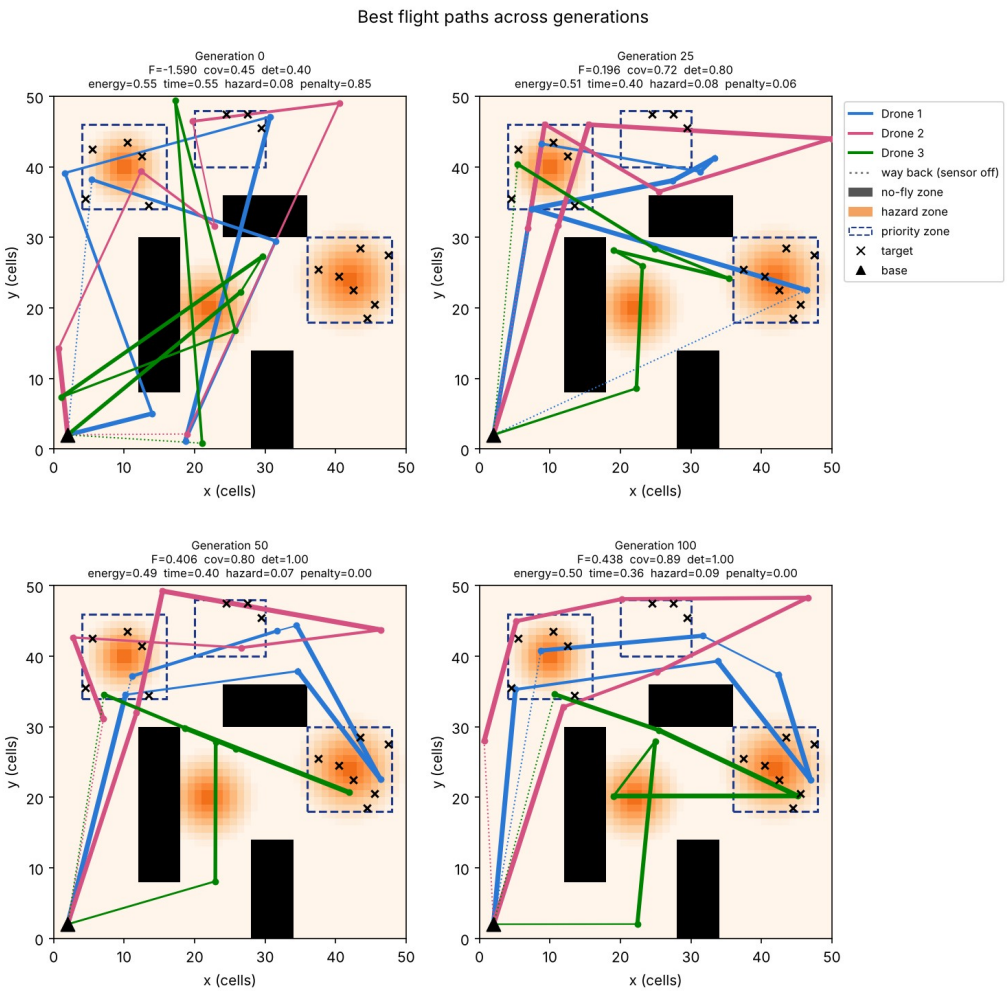


Figure 2: Best flight paths in generations 0, 25, 50 and 100 (thicker line = stronger sensor, dotted = way home)

In generation 0 the routes are random zig-zags that cross the no-fly zones (black). Over the generations the routes move around the no-fly zones, each drone takes its own area, and the strong sensor (thick lines) is used inside the priority zones (blue boxes). The coverage heatmaps show the same: the covered area moves to the important zones.

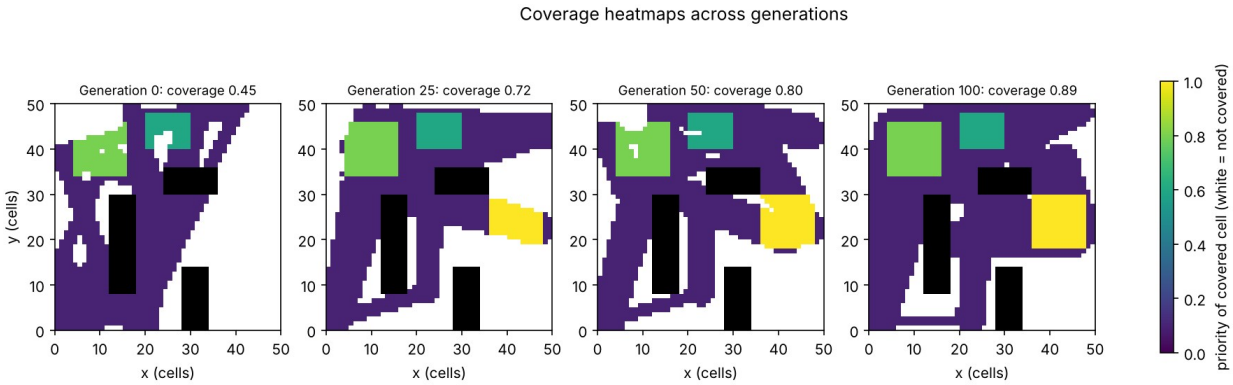


Figure 3: Coverage of the best plan across generations (colour = priority of the covered cell, white = not covered)

6. Hyperparameter Sensitivity Analysis

I changed one setting at a time and kept the others at their default values. Each setting was run with 3 seeds for 100 generations. Table 3 shows the final best fitness (mean ± standard deviation).

Table 3: Final best fitness for different settings (3 seeds)

Setting	Value 1	Value 2	Value 3
Population size	20: 0.226 ± 0.145	50: 0.394 ± 0.054	100: 0.443 ± 0.006
Mutation rate	0.02: 0.240 ± 0.028	0.1: 0.394 ± 0.054	0.3: 0.409 ± 0.003
Tournament k	2: 0.418 ± 0.008	3: 0.394 ± 0.054	5: 0.298 ± 0.099

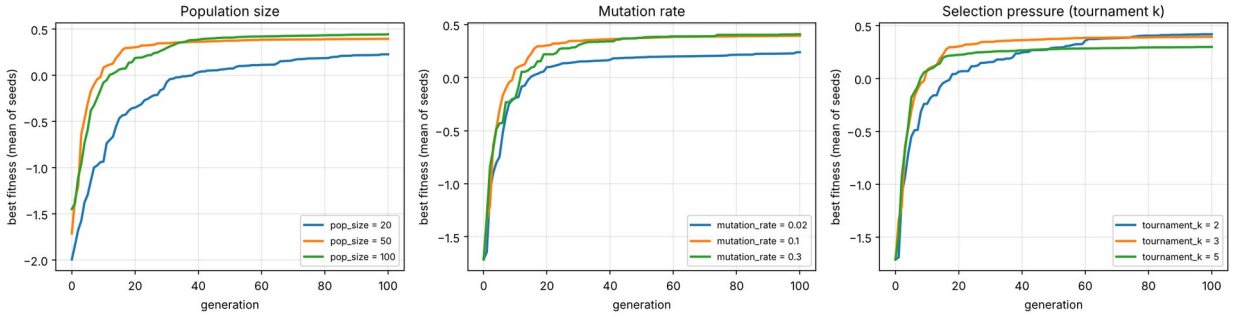


Figure 4: Mean best fitness for each setting

Population size: a bigger population gives better and more stable results, because it searches more of the map at the same time (more exploration). The cost is more fitness evaluations: 100 plans need twice the time of 50 plans.

Mutation rate: 0.02 was clearly the worst. With so little mutation the population becomes similar very quickly and gets stuck (premature convergence). 0.1 and 0.3 both worked well; 0.3 was slightly better and more stable here, because the routes need many changes to move around the no-fly zones.

Selection pressure: a small tournament (k = 2) gave the best result, and k = 5 the worst. With a large k the best plans are chosen almost every time, the population loses diversity and some runs get stuck early. This shows the balance between exploration (low pressure) and exploitation (high pressure).

7. GA versus Random Search

Random search creates random plans (with the same repair) and keeps the best one. To be fair it got exactly the same number of fitness evaluations as the GA (4850). Both were run with 5 seeds.

Table 4: GA vs. random search (mean ± standard deviation over 5 seeds)

Measure (best plan)	Genetic Algorithm	Random search
Fitness	0.415 ± 0.050	-0.756 ± 0.119
Coverage	0.888 ± 0.023	0.665 ± 0.071
Detection	1.000 ± 0.000	0.667 ± 0.140
Energy	0.492 ± 0.041	0.503 ± 0.032
Time	0.401 ± 0.051	0.509 ± 0.044
Hazard	0.059 ± 0.014	0.072 ± 0.013
Penalty	0.011 ± 0.023	0.509 ± 0.045

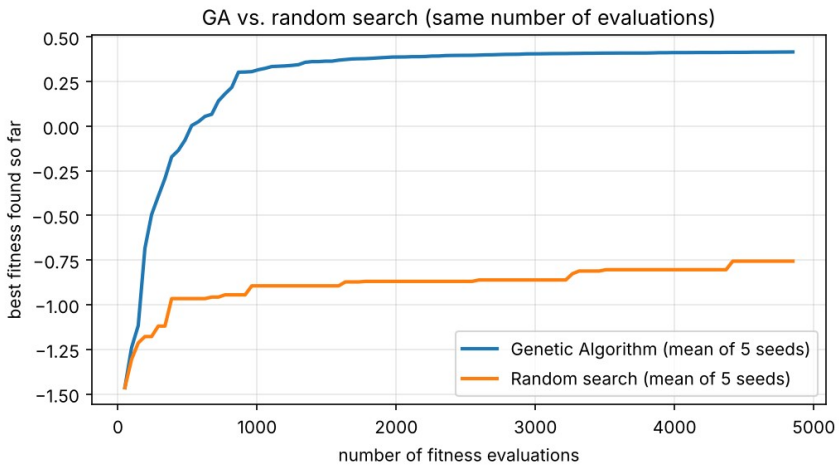


Figure 5: Best fitness found so far vs. number of evaluations

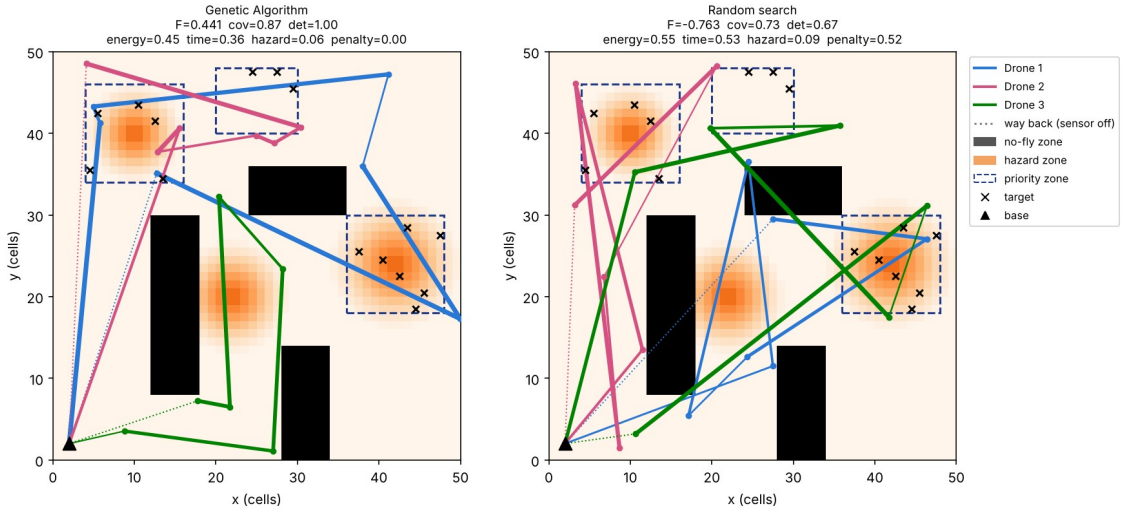


Figure 6: Best plan of the GA (left) and of random search (right), seed 1

The GA is much better than random search. Random search never found a plan that avoids all no-fly zones, so it always has a penalty; 4 of the 5 GA runs broke no rule at all. What makes the GA solution special: (1) structured paths: each drone flies to its own area and the routes go around the no-fly zones instead of random zig-zags; (2) sensor scheduling: the strong sensor is used over the priority zones, a weaker one on the way there; (3) priority focus: higher coverage and all targets found; (4) shorter mission time and slightly less hazard. Random search only finds such a plan by luck, while the GA builds it step by step from good parts of earlier plans.

8. Conclusion and Limitations

I built a simple GA that plans the routes, speeds and sensor power of a drone fleet. The real-valued chromosome, the weighted fitness function, custom crossover and mutation, and repair plus penalty together give good and legal plans that are far better than random search. Limitations: the map is small and fixed, the energy and sensor models are simple, and the weighted sum gives only one compromise. In the future the GA could use a moving hazard field, more drones, and a multi-objective method such as NSGA-II to show many trade-offs at once.